IoT Cat Feeder

An automatic, portion-controlled cat feeder with a web dashboard on the ESP8266

Software Design Description

Chindanai Jaiman 662115012
Bachelor of Science Software Engineering Program
College of Arts, Media, and Technology, Chiang Mai University

November 2026

Project Advisor
[Advisor Name]

Table of Contents

Chapter 1 Introduction **3**
1.1 Purpose 3
1.2 Scope 3
1.3 Design Overview 3
1.4 Definitions 3
Chapter 2 System Architecture **4**
2.1 Architectural Overview 4
2.2 Context View 4
2.3 Component View 5
2.4 Deployment and Power Design 6
2.5 Design Rationale 6
Chapter 3 Detailed Design **7**
3.1 Data Design 7
3.2 Component Description 7
3.3 State Design 8
3.4 Interaction Flows 9
3.5 Interface Design 9
3.6 Timing and Resource Budget 10
3.7 Error Handling and Recovery 10
3.8 Security Considerations 11
Chapter 4 Traceability and Verification **12**
4.1 Requirements Traceability Matrix 12
4.2 Test Plan 12
4.3 Known Limitations 12
4.4 Future Work 13
Appendix A Design Review Checklist **14**

Chapter 1 Introduction

1.1 Purpose

This document describes how the IoT Cat Feeder is built. It is the design counterpart to the Software Requirements Specification, and it is organised around the design views named in IEEE Std 1016: decomposition, dependency, interface, and detail. Every design entity below traces back to a requirement, and every requirement traces forward to a test.

The reader this document serves is the next student who has to rebuild the device, or the examiner who has to decide whether the code and the circuit are one system or two unrelated artefacts.

1.2 Scope

The design covers the firmware in `firmware/feeder/feeder.ino`, the wiring in `docs/wiring_nodemcu_servo.html`, and the power arrangement. It excludes the mechanical construction of the cardboard hopper beyond the flap geometry, which is described only where it constrains the servo.

1.3 Design Overview

The firmware is one translation unit with no classes, no threads, and no dynamic allocation after start-up. It runs a single cooperative loop that serves HTTP, watches the clock, and closes the flap. Each of those three duties is a function call, and none of them blocks.

The single organising rule is that all feeding passes through `feed(source)`. Three triggers converge on it: the HTTP handler, the scheduler, and the presence detector. Counting, safety, and the daily cap therefore live in one place instead of three.

1.4 Definitions

Table 1. Design terms

Term	Meaning in this document
Component	A named function or small group of functions with one responsibility.
State layer	The `struct Feeder` instance and the functions that read or write it.
Minute key	Integer `(day * 100 + hour) * 100 + minute`, used to detect a new minute.
Flap cycle	One open command followed by the automatic close.
Target feature	Designed here, implemented only if the parts pass test before 2 November 2026.

Chapter 2 System Architecture

2.1 Architectural Overview

The system has three layers. The network layer terminates HTTP and converts requests into commands. The control layer decides whether a meal should happen. The device layer drives the servo and reports electrical state. The layers are functions in one file rather than modules in separate files, because the whole program is 259 lines and a file split would add build complexity without adding clarity.

Table 2. Layer responsibilities

Layer	Components	Responsibility
Network	keepWifi, server, request handlers	Maintain the link, parse requests, serialise responses.
Control	feed, checkSchedule, markFeed, presence detector (target)	Decide when a meal occurs and record it exactly once.
Device	Servo, closeFlap, sensor driver (target)	Drive the actuator and sample the sensors.
Presentation	PAGE, dashboard script	Render state and send one command.

2.2 Context View

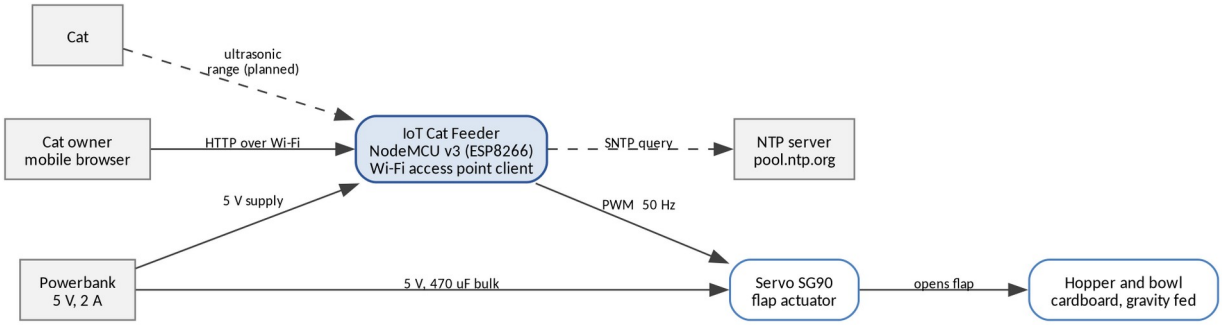


Figure 1. Context view: external entities and energy flow.

2.3 Component View

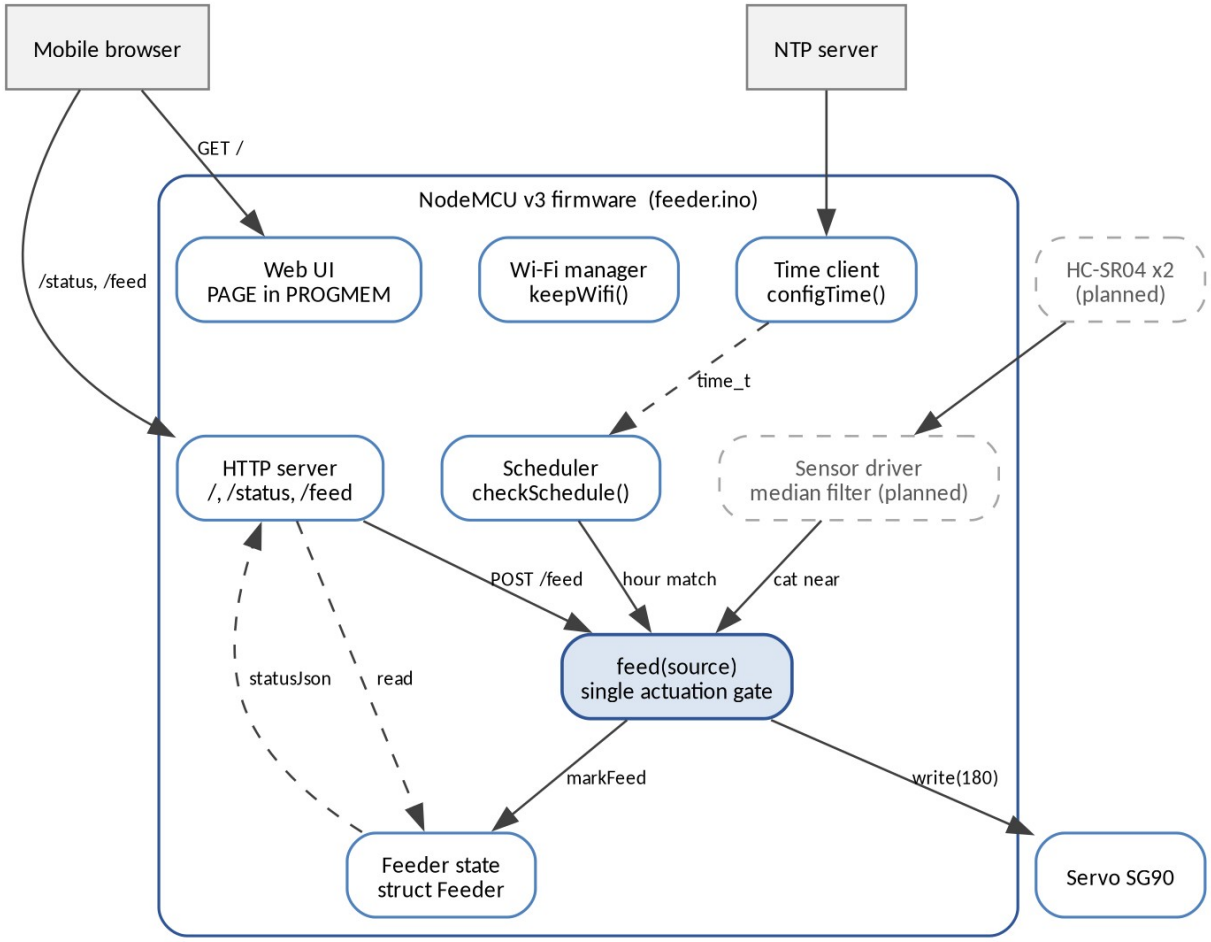


Figure 2. Component view. Dashed components are target features.

Table 3. Component catalogue

Component	Kind	Depends on	Status
PAGE	Constant string in PROGMEM	Nothing	Delivered
server	ESP8266WebServer instance	PAGE, statusJson, feed	Delivered
statusJson	Function	Feeder, clockText, scheduleText, WiFi	Delivered
feed	Function	markFeed, Servo, millis	Delivered
closeFlap	Function	Servo, Feeder	Delivered
markFeed	Function	time, Feeder	Delivered
checkSchedule	Function	time, Feeder, feed	Delivered
clockText, scheduleText	Functions	time, constant table	Delivered
keepWifi	Function	WiFi, millis	Delivered
Presence detector	Function, target	Ultrasonic driver, feed	Target
Mass controller	Function, target	HX711 driver, feed	Target

2.4 Deployment and Power Design

The device has one power input and one ground reference. The servo is the only load that draws significant current, and it is the reason the power design differs from the obvious one.

Table 4. Power distribution

Rail	Source	Loads	Notes
5 V	Powerbank, 2 A	VIN of the board, servo V+	Servo must not be fed from 3V3.
3.3 V	On-board regulator	ESP8266 core, logic	Limited to a few hundred milliamps.
Ground	Common	Board, servo, sensors	A single ground reference for PWM to be valid.
Bulk capacitance	470 uF, 16 V	Across 5 V and ground	Absorbs the servo inrush.

The failure that forced this design was observed during bring-up. With the servo powered from the notebook USB port, the board reset whenever the servo started. The stall current of the servo pulls the rail below the regulator dropout. Two changes fixed it: a powerbank with a higher current limit, and a 470 uF capacitor next to the servo connector. The bring-up sketch firmware/servo_test/servo_test.ino exists so that this behaviour can be reproduced without the rest of the firmware.

2.5 Design Rationale

Table 5. Design decisions and alternatives

Decision	Alternative	Why the alternative lost
Single file, functions only	Split modules with headers	259 lines. A split adds include order problems for no gain.
Cooperative loop, no delay	delay(900) after opening the flap	delay blocks the HTTP server. The page would freeze on every meal.
Close driven by millis() in the loop	Close driven by the HTTP handler	The handler runs only if the client stays connected. The loop always runs.
One feed(source) gate	Separate functions per trigger	Three triggers would need three copies of the counting and cap logic.
Embedded HTML in PROGMEM	SPIFFS file, CDN assets	No file system to corrupt, no network fetch, one flash image.
Boolean state for the flap	Enum state machine	Two states in the delivered scope. The enum becomes justified when the target features land.
In-RAM history	Flash record, external database	The owner asks one question per day. A flash record is scheduled, a database is not justified.

Chapter 3 Detailed Design

3.1 Data Design

The program holds one state record and three constants. There is no heap use after start-up, apart from the String growth inside the JSON builder, which is bounded by the buffer size.

Table 6. State record

Field	Type	Initial	Meaning
open	bool	false	Flap is currently open.
openedAt	unsigned long	0	millis() at the last open command.
countToday	int	0	Meals since the last local date change.
lastDay	int	-1	Day of month that countToday belongs to.
lastTime	String	-	HH:MM of the last meal.
lastSource	String	empty	Trigger of the last meal.

Table 7. Constants

Name	Value	Used by
SERVO_PIN	D5, GPIO14	setup
CLOSED_DEG	0	closeFlap, setup
OPEN_DEG	180	feed
OPEN_MS	900	loop
SCHEDULE_HOUR	8, 13, 18	checkSchedule, scheduleText
SCHEDULE_MINUTE	0	checkSchedule
TZ_OFFSET	25200 s	setup

OPEN_DEG is the commanded angle, not the delivered angle. The servo in use reaches about 95 degrees before the flap linkage stops it against the hopper. The mechanism, not the code, is the real limit. A different servo requires this value to be re-measured.

3.2 Component Description

Table 8. Function catalogue

Function	Signature	Purpose
setup	void setup()	Attach the servo, close the flap, start Wi-Fi, start the time client, register routes, start the server.
loop	void loop()	Poll HTTP, enforce the close deadline, check the schedule, keep the link.
feed	void feed(const String &source)	Record the meal, open the flap, stamp the open time.
closeFlap	void closeFlap()	Command the closed angle and clear the open flag.
markFeed	void markFeed(const String &source)	Update count, time, source, and handle the date change.
checkSchedule	void checkSchedule()	Compare local time with the meal table, at most once per minute.
clockText	String clockText()	Format the local time, or a placeholder before NTP.
scheduleText	String scheduleText()	Format the meal table for the dashboard.

Function	Signature	Purpose
statusJson	String statusJson()	Build the eight-field status object.
keepWifi	void keepWifi()	Reconnect at most once every 5 s, with one console line.

3.2.1 Algorithm: the minute key

The scheduler must fire once per scheduled minute, but loop runs thousands of times per second. The design collapses the date and time into one integer and compares it with the value from the previous iteration.

key = (tm_mday * 100 + tm_hour) * 100 + tm_min

- A repeated call inside the same minute produces the same key, so the function exits before the meal table is examined.
- The key changes at a minute boundary, at an hour boundary, and at a month boundary, because the day field is included. Two identical keys therefore indicate the same minute, not merely the same clock face.
- The design trades a rare false negative for a common false positive. A clock that jumps backward one minute re-enters a previous key and is suppressed. A clock that jumps backward by hours may re-enter a key that matches a meal time, which is the residual risk recorded in Section 3.7.

3.2.2 Algorithm: median filtering for ranging (target)

A single ultrasonic reading on a kibble surface is unreliable because the beam scatters. The target design takes ten samples, sorts them, and uses the median. A detection is confirmed only after three consecutive medians fall inside the range window, which removes one-off echoes from the hopper wall or the cat's own movement.

3.3 State Design

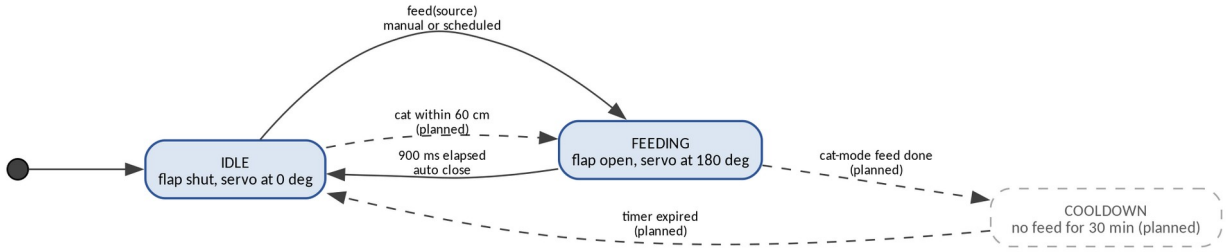


Figure 3. State design. Solid transitions are delivered, dashed are targets.

Table 9. State transitions

From	Event	Guard	To	Action
IDLE	Boot	None	IDLE	Servo to closed angle.
IDLE	feed(source)	None	FEEDING	Record meal, servo to open angle, stamp time.
FEEDING	Loop iteration	millis() - openedAt >= 900	IDLE	Servo to closed angle.
FEEDING	Loop iteration	Close deadline not reached	FEEDING	None.
IDLE	Cat within 60 cm, three times	Cooldown expired, cap not reached	FEEDING	Record meal with source presence.
FEEDING	Presence meal completed	None	COOLDOWN	Start 30 minute timer.
COOLDOW	Loop iteration	30 minutes elapsed	IDLE	None.

From	Event	Guard	To	Action
N				

3.4 Interaction Flows

3.4.1 Flow: manual feeding

1. The dashboard sends POST /feed with an empty body.
2. server.handleClient() in the main loop dispatches the request to the registered handler.
3. The handler calls feed("manual") and sends {"ok":true}.
4. feed calls markFeed, which reads time(), formats HH:MM, resets the count if the day changed, increments the count, and stores the source.
5. feed writes the open angle and records millis().
6. On a later loop iteration, the close condition becomes true and closeFlap writes the closed angle.
7. The dashboard, polling every 2 seconds, reads open: false and the updated count.

3.4.2 Flow: scheduled feeding

1. A loop iteration calls checkSchedule.
2. The function reads time(). If the value is below the validity epoch, it returns.
3. It computes the minute key and compares it with the stored key. If equal, it returns.
4. It stores the new key and compares the hour and minute with each row of the meal table.
5. On a match it calls feed("schedule") and returns, so a second match in the same iteration cannot occur.

3.4.3 Flow: network recovery

1. Each loop iteration calls keepWifi.
2. If the link is up and the address has not been printed since the last recovery, the address is printed once.
3. If the link is down, the function returns unless 5 seconds have passed since the last attempt.
4. It starts a connection attempt and stamps the time. The attempt does not block.

3.5 Interface Design

3.5.1 HTTP contract

Table 10. Endpoint contracts

Endpoint	Method	Success	Failure
/	GET	200, HTML page	404 for other methods
/status	GET	200, JSON, eight fields	404 for other methods
/feed	POST	200, {"ok":true}	404 for other methods
Any other path	Any	Not applicable	404, plain text

3.5.2 Response format

The status object is built with snprintf into a 260 byte buffer. The buffer is adequate for the current field lengths and has no overflow check. If a source string grows, the tail of the object is truncated silently. This is recorded as a limitation rather than fixed, because the source values are compile-time constants.

3.5.3 User interface

Table 11. Dashboard layout

Region	Content	Interaction
Header	Device name and one-line purpose	None
Status card	Five rows: flap, meals today, last meal, device time, schedule	Read only
Action	Full-width feeding button	POST /feed, then a 1300 ms lockout
Footer	RSSI and IP, or a lost-connection message	None

3.6 Timing and Resource Budget

The values below are a design budget, not measurements. The measured columns are filled during the test on 3 November.

Table 12. Timing budget

Path	Budget	Reason
HTTP request to response	40 ms	One snprintf and one send on a local network.
Button press to flap moving	150 ms	Network round trip plus servo dead band.
Servo travel, closed to open	200 ms	About 95 degrees at SG90 rated speed.
Flap open dwell	900 ms	Fixed by OPEN_MS, long enough for a portion to fall.
Total flap cycle	1.1 to 1.3 s	Travel plus dwell plus close travel.
Dashboard refresh	2000 ms	setInterval in the page.
Schedule detection delay	Under 1 s	The loop interval bounds the check.
Main loop period	Under 5 ms	Four non-blocking calls.

3.7 Error Handling and Recovery

Table 13. Failure modes

Failure	Detection	Response	Residual risk
Wi-Fi lost	WiFi.status()	Retry every 5 s without rebooting	Dashboard unavailable until the link returns
Time not yet valid	Epoch below 1600000000	Scheduler disabled, placeholder clock shown	A meal during the first seconds after boot is skipped
Clock jumps backward	Repeated minute key	The meal in that minute is suppressed	A jump of hours can re-enter a meal minute
Client disconnects mid-request	Handler never returns	The loop closes the flap on schedule	The count is already incremented
Servo stalls	Not detected	None in firmware	Brownout, reboot, lost meal
Unknown path	onNotFound	404 with no state change	None
Buffer truncation	Not detected	None	Malformed JSON if a field grows

3.8 Security Considerations

- The trust boundary is the Wi-Fi network. Any client on it can dispense food. This is accepted for a single household and is documented in the SRS limitations.
- No administrative endpoint exists, so there is no credential to steal from the device.
- The Wi-Fi credentials live in `secrets.h`, which is excluded by `.gitignore`. The repository ships `secrets.example.h` only.
- The device sends no telemetry outbound except the NTP query, which carries no user data.

Chapter 4 Traceability and Verification

4.1 Requirements Traceability Matrix

Table 14. Traceability

Requirement	Component	Test
FR-01, FR-02, FR-03	checkSchedule, configTime	TC-04, TC-05
FR-04, FR-05	feed, closeFlap, loop	TC-02, TC-03
FR-06, FR-12	Route registration, onNotFound	TC-08
FR-07, FR-10, FR-11	statusJson, PAGE	TC-01, TC-06
FR-08	Dashboard script	TC-01
FR-09	markFeed	TC-07
FR-13, FR-14	Presence detector, target	TC-09
FR-15, FR-16	Mass controller, target	TC-10
FR-17, FR-18	Not implemented	Future work
NFR-RL-01	loop close condition	TC-03
NFR-RL-02	keepWifi	TC-11
NFR-SEC-01	.gitignore	Inspection

4.2 Test Plan

Table 15. Test cases

ID	Case	Procedure	Pass condition
TC-01	Dashboard loads	Open the device address on a phone.	Five rows populated within 2 s.
TC-02	Manual feeding	Press the button, observe the flap.	Flap opens within 1.5 s.
TC-03	Automatic close	Press 50 times with 5 s between presses.	All 50 close without intervention. No flap stays open.
TC-04	Schedule fires	Set the next meal one minute ahead and wait.	One meal at the minute, recorded with source schedule.
TC-05	Duplicate suppression	Observe the same minute for 60 s.	Exactly one meal.
TC-06	Status accuracy	Compare every field with the device state.	All eight fields match.
TC-07	Date rollover	Let the device run past local midnight.	Count returns to zero.
TC-08	Method rejection	GET /feed, POST /status, and an unknown path.	404 for each, with no meal dispensed.
TC-09	Presence detection	Walk past the sensor 20 times at 50 to 70 cm.	At least 16 detections, no meal inside the cooldown.
TC-10	Portion mass	Dispense 10 meals onto a scale.	Each within 15 percent of target.
TC-11	Network recovery	Power cycle the router.	The device rejoins within 30 s, no reboot.

4.3 Known Limitations

Table 16. Limitations accepted in version 1.0.0

ID	Limitation	Impact	Planned fix
L-01	History is in RAM	A reboot clears the count	Flash record, FR-17

ID	Limitation	Impact	Planned fix
L-02	No authentication	Any client on the network can feed	Token, or network isolation only
L-03	Count increments before the servo moves	A failed servo still counts a meal	Count after the close, or verify with the load cell
L-04	Status buffer is not overflow checked	Possible truncated JSON	Larger buffer or a structured writer
L-05	No external watchdog	A hang stops feeding until the power is cycled	Hardware watchdog timer
L-06	Single hopper, single cat	No identity or separation	RFID collar tag

4.4 Future Work

- Persist the last 32 meals to flash with an append-only record.
- Add an ESP32-CAM module and store one still image per meal on the device.
- Identify the cat with an RFID collar tag so a second cat cannot take the portion.
- Add a hardware watchdog so a firmware hang recovers without the owner.
- Replace the cardboard body with a laser-cut enclosure that fixes the servo geometry.
- Publish state to an MQTT broker when the deployment network allows port 1883.

Appendix A Design Review Checklist

Table 17. Design review

Check	Result
Every requirement maps to a component	Yes, Section 4.1
Every component has one responsibility	Yes, Section 2.3
No blocking call in the main loop	Yes, delay is used only in the bring-up sketch
The flap has a bounded open time in every path	Yes, the loop enforces it independent of HTTP
The failure of a client does not change device state	Yes, Section 3.7
Electrical limits of each pin are respected	Partly. The echo divider is required for the target sensors and is recorded in the SRS limitations
The design states what it does not do	Yes, Section 4.3